

6. Semaphores and Mutexes

Syllabus Coverage: Gateway Classes Slides 60–75 | Counting vs Binary Semaphores, Block-Wakeup Implementation, Mutex vs Semaphore.

6.1 Semaphore Concept & Operations

An integer variable accessed strictly via atomic `wait()` and `signal()` operations.

- **Binary Semaphore:** Value restricted to 0 or 1. Used for mutual exclusion.
- **Counting Semaphore:** Value 0 to N. Used for tracking pools of finite resources.

Semaphores & Mutexes Architecture

Counting vs Binary Semaphores & Block-Wakeup Queue Implementation

Semaphore Types & Definitions

1. Binary Semaphore (Value: 0 or 1)

- Used for Mutual Exclusion (Locking).
- Similar to Mutex, but lacks thread ownership.

Initial value = 1 (Unlocked)

2. Counting Semaphore (Value: 0 to N)

- Used for resource management (Pool of N units).
- E.g., Database connection pool, 5 printers.

Initial value = N (Available resource count)

Block & Wakeup (No Busy Waiting)

```
typedef struct {
    int value;
    struct process *list; // Waiting PCB queue
} semaphore;

wait(S) { S.value--;
    if (S.value < 0) { add to S.list; block(); } }

signal(S) { S.value++;
    if (S.value <= 0) { wakeup(P from list); } }

[S.value] when negative = Count of waiting processes!
```

Mutex vs Binary Semaphore: Crucial Technical Differences

Feature	Mutex (Mutual Exclusion Lock)	Binary Semaphore
Ownership	Has ownership: Jo lock karega, wahi unlock karega.	No ownership: Koi bhi process signal() kar sakta hai.
Purpose	Strictly Locking mechanism (Critical Section).	Signaling mechanism (Coordination & Order).
Priority Inversion	Supports Priority Inheritance Protocol.	Does not support Priority Inheritance.

6.2 Block & Wakeup Implementation (Avoiding Busy Waiting)

```
typedef struct {
    int value;
    struct process *list; // Queue of waiting PCBs
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add_to_queue(S->list, current_process);
        block(); // Process goes to sleep; CPU freed!
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        process *P = remove_from_queue(S->list);
    }
}
```

```
        wakeup(P); // Wakes up sleeping process to Ready queue
    }
}
```

6.3 Mutex vs Binary Semaphore Comparison

Feature	Mutex	Binary Semaphore
Mechanism	Strict Locking mechanism.	Signaling / Notification mechanism.
Ownership	Locked thread owns it; only owner can unlock.	No ownership; any process can signal.
Priority Inheritance	Supported (prevents priority inversion).	Not supported natively.